CASE STUDY 79

# ReviewRadar

## REVIEW HELPFULNESS ANALYSIS

Machine Learning Case Study - 79 • Final Documentation

Mahesh Rajpurohit · 150096724003 · Elon Musk

A concise, explainable system for estimating how useful a review sentence is to a shopper.

| TASK | DATA | MODEL | OUTPUT |
|---|---|---|---|
| **Review helpfulness** | **20,000 + 2,000** | **TF-IDF + Ridge** | **0–2 score** |
| Estimate | Train / test | Regression | 3 quality bands |

**WHAT THIS REPORT COVERS**

Problem definition, dataset decisions, EDA, preprocessing, model development, evaluation, Streamlit application and submission evidence — with a compact project workflow added for quick understanding.

# 1 Problem Definition

Online shopping contains a very large amount of review text, but not every sentence contributes useful information to a purchase decision. The objective is to estimate the helpfulness of an individual review sentence from its text and product context.
The final system produces a continuous score from 0 to 2 and maps it into three easy-to-read bands: Needs More Detail, Helpful, and Highly Helpful.

# 2 Dataset & Documentation

The project uses Amazon's public Helpful Sentences from Reviews dataset. The benchmark was created from 45,091 Amazon reviews for 123 products across six categories. The released benchmark contains 20,000 training sentences and 2,000 test sentences. Human annotators rated each sentence on a 0–2 scale for usefulness in making a purchase decision.
The data is distributed through an Amazon-managed public S3 bucket listed in the AWS Open Data Registry. The key fields used are product identifier, sentence text, product title and helpfulness score. The original benchmark paper reports that sentence length and sentiment are associated with helpfulness, while also noting that helpfulness is subjective.

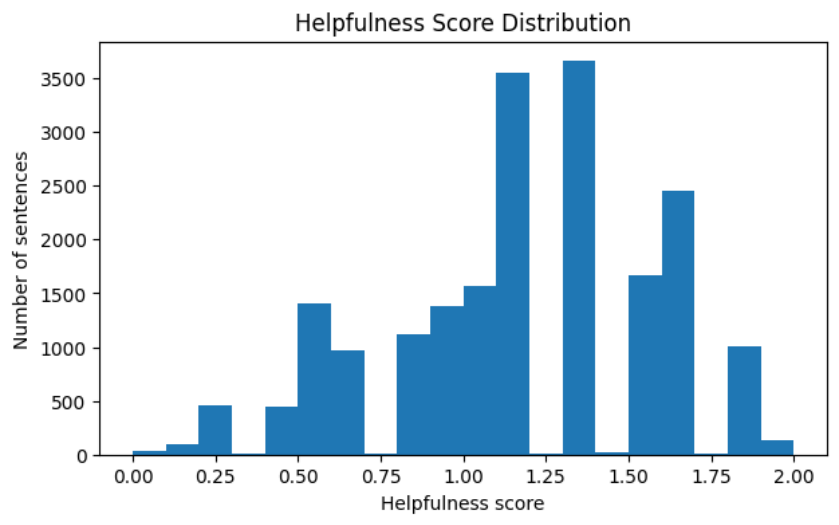
| Field | Role |
|---|---|
| Product identifier | Groups sentence with its product context |
| Sentence text | Primary text feature |
| Product title | Context feature / source metadata |
| Helpfulness score | Continuous target: 0–2 |

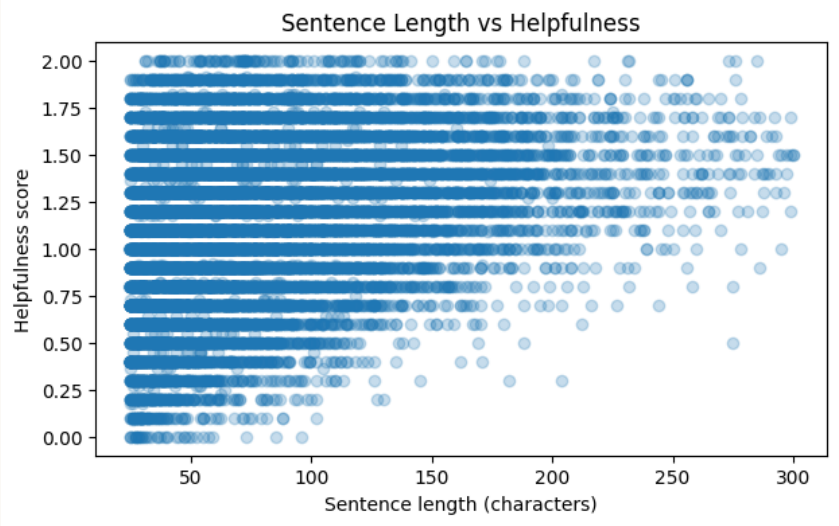
**DATASET DECISION**

The official 20,000/2,000 train-test benchmark is preserved rather than creating an arbitrary split. This makes the experiment easier to reproduce.

# 3 Exploratory Data Analysis

Exploratory analysis focuses on the distribution of helpfulness scores, sentence length, and the relationship between length and helpfulness. The dataset has a clear mode around 1.3 and the original study reports a positive relationship between sentence length and helpfulness.
These checks help confirm that the target is continuous and support using a regression model rather than forcing the task into an artificial binary label.



# 4 Preprocessing

Missing text is removed, HTML markup is cleaned, whitespace is normalized, and duplicate sentence/product combinations are removed. The official 20,000/2,000 train-test split is preserved rather than creating an arbitrary new split.

| Step | Decision |
| --- | --- |
| Cleaning | Remove missing / malformed text and normalize whitespace |
| Text preparation | Clean HTML and combine useful textual context |
| Feature extraction | TF-IDF converts text into numeric features |
| Split | Use the supplied train / test benchmark |

# 5 Model Development

Two experiments are used: a simple mean-value baseline and Ridge Regression on TF-IDF text features. The baseline checks whether the ML model learns more than simply predicting the average helpfulness score. Ridge is the final model because helpfulness is continuous and TF-IDF creates a high-dimensional sparse feature space. Ridge adds regularization, trains quickly and keeps the implementation small and explainable. The original benchmark also reports TF-IDF + Ridge as a lightweight model, supporting this choice.

| Experiment | Why it is included |
|---|---|
| Mean baseline | Reference point: predicts the average score for every sentence |
| TF-IDF + Ridge | Learns text patterns while controlling overfitting with L2 regularization |

**ADDITIONAL PROJECT WORKFLOW**

DATA → CLEAN → EDA → TF-IDF → BASELINE → RIDGE → METRICS → STREAMLIT

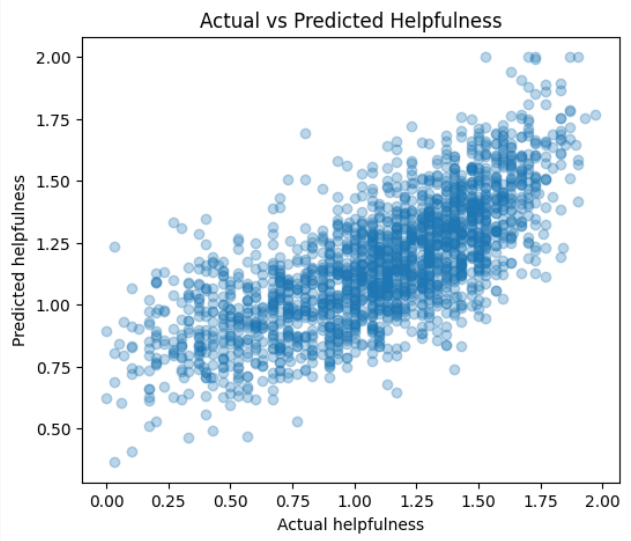
# 6 Evaluation

The primary measures are Mean Absolute Error (MAE), Mean Squared Error (MSE), $R^2$ and Pearson correlation. These directly measure how close predicted helpfulness scores are to human ratings and whether the model preserves their ordering.

For a simple percentage-style result, the project also reports “accuracy within ±0.5”, meaning the percentage of test predictions whose absolute error is no greater than 0.5 helpfulness points. This is supplementary to, not a replacement for, the standard regression metrics.

| Metric | What it tells us |
|---|---|
| MAE | Average absolute distance from the human score; lower is better |
| MSE | Penalizes larger prediction errors more strongly; lower is better |
| $R^2$ | How much target variance is explained by the model; higher is better |
| Pearson correlation | How strongly predicted and human scores move together; higher is better |
| ±0.5 accuracy | Share of predictions within half a helpfulness point |

```
Baseline MAE: 0.328
Ridge MAE: 0.223
MSE: 0.084
R2: 0.487
Pearson correlation: 0.723
Accuracy within ±0.5: 91.3%
```

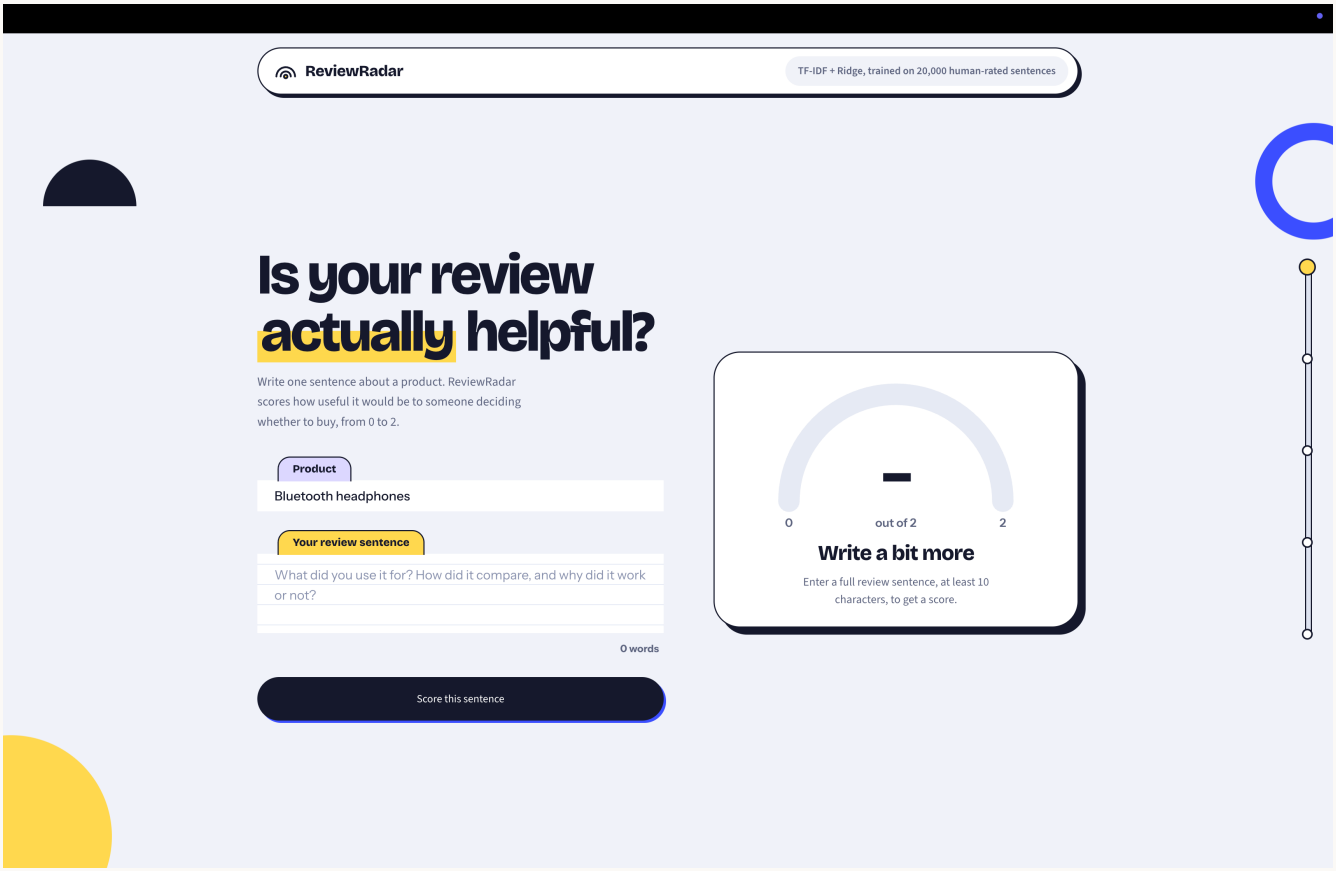



**RESULT DISCIPLINE**

The notebook calculates the final scores from the official held-out test set. No performance value is manually hard-coded into the report or Streamlit interface.

# 7 Streamlit Application

ReviewRadar turns the trained model into an interactive interface. The user enters a product/context and review sentence, then receives a helpfulness score, a clear quality band and the benchmark metrics.
The interface uses a dark editorial dashboard, rounded cards, compact metric blocks, quick-test examples and a visual score meter rather than a generic chatbot layout. The application trains the model on its first run and caches the resulting artifact for later predictions.



# 8 Final Report & Presentation

The notebook provides the reproducible implementation, graphs, model training and evaluation outputs. The Streamlit application demonstrates the final solution. The report is intentionally concise so the key problem, data decisions, model choice, results and limitations can be presented clearly during submission and viva.

# References


- Amazon Web Services Open Data Registry — Helpful Sentences from Reviews: https://registry.opendata.aws/helpful-sentences-from-reviews/
- Gamzu, I., Gonen, H., Kutiel, G., Levy, R., & Agichtein, E. (2021). Identifying Helpful Sentences in Product Reviews. NAACL-HLT, 678–691: https://aclanthology.org/2021.naacl-main.55/
- Project implementation: ReviewRadar notebook, model_utils.py and app.py.


**TAKEAWAY**

Problem: estimate review helpfulness. Data: 20k train + 2k test human-rated sentences. Model: TF-IDF + Ridge Regression. Evaluation: MAE, MSE, $R^2$, Pearson, ±0.5 accuracy. Product: interactive Streamlit prototype.